

Лекция 3. Анализ сетевого трафика: tcpdump и Wireshark

Цель лекции

Понять, когда пакетный анализ помогает в диагностике Linux-сервера

Выбирать точку захвата, интерфейс и режим видимости трафика

Различать capture filters tcpdump и display filters Wireshark

Записывать, читать и безопасно хранить PCAP-файлы

Интерпретировать ARP, DNS, TCP, HTTP и ограничения HTTPS

Учебные вопросы

1. Анализ трафика, точка захвата, интерфейс и promiscuous mode
2. Capture filters и display filters; фильтры host, net, port, protocol, src, dst
3. PCAP: запись, чтение и безопасное хранение
4. ARP и DNS
5. TCP handshake, завершение, RST, retransmission, duplicate ACK и timeout
6. HTTP и ограничения анализа HTTPS

Пакетный анализ показывает фактический обмен, а не предположен

tcpdump - консольная утилита Linux для захвата и просмотра сетевых пакетов

Wireshark - графический анализатор сетевого трафика и PCAP-файлов

Захват помогает отличить закрытый порт от молчаливой потери пакетов

Он показывает, уходит ли запрос с узла и приходит ли ответ

Пакетный анализ не заменяет ip route, ss и journalctl, а дополняет обычную диагностику

Особенно полезны захваты для ARP, DNS, TCP, HTTP.

Точка захвата определяет, какие факты попадут в PCAP

Capture Point - место, где выполняется захват трафика

На клиенте видно, что отправляет и получает клиент

На сервере видно, дошел ли запрос до серверного интерфейса

На маршрутизаторе или firewall видно прохождение между сетями

На промежуточном устройстве можно увидеть, где пакет проходит и где пропадает

Неверная точка захвата может создать ложный вывод о причине проблемы

Двухточечный захват помогает локализовать потерю

Если запрос виден на клиенте, но не виден на сервере, потеря находится между ними

Если запрос пришел на сервер, но ответа нет, проверяется приложение или локальный firewall сервера

Если сервер отправил ответ, а клиент его не получил, проверяется обратный маршрут и асимметрия

Если ответ виден на firewall, но не виден на клиенте, проблема ближе к клиентской сети

Два захвата дают больше доказательств, чем один
PCAP в спорной точке

Интерфейс выбирают по локальному решению маршрутизации

Команда `ip -br link` показывает сетевые интерфейсы и их состояние

Команда `ip route get 192.168.30.10` показывает локальный выбор пути ядром Linux

Пример вывода: `192.168.30.10 via 192.168.10.1 dev enp0s3 src 192.168.10.20`

Из вывода видны назначение, `next-hop`, интерфейс и `source IP`

`ip route get` не доказывает, что пакет достигнет назначения

После выбора интерфейса можно запускать `tcpdump`.

Promiscuous Mode расширяет прием кадров только на своем интерфейсе

Promiscuous Mode - режим, в котором интерфейс передает ОС все кадры, физически или виртуально пришедшие на него

Интерфейс принимает кадры даже тогда, когда Destination MAC не совпадает с его MAC-адресом
Этот режим не заставляет коммутатор отправлять чужой unicast-трафик на порт сервера

В обычной коммутируемой сети узел не становится анализатором всего сегмента

Для чужого трафика нужны SPAN, Port Mirroring, TAP, захват на маршрутизаторе или захват на нужной VM

В VirtualBox видимость трафика зависит от режима сети

В NAT и NAT Network гостевая ОС видит не то же самое, что физический адаптер хоста

В Bridged Adapter видимость зависит от физической сети и политики гипервизора

В Internal Network и Host-Only трафик ограничен виртуальным сегментом

Настройка Promiscuous Mode виртуального адаптера влияет на то, что гипервизор передаст гостевой ОС

Разрешение promiscuous в Linux не гарантирует, что VirtualBox отдаст чужие кадры

Если ожидаемых пакетов нет, сначала проверяется

Схема: выбор точки захвата в серверной сети

Правильная точка захвата = правильные факты в PCAP

Легенда

●

 Хорошие точки захвата

○

 Дополнительные точки

⊗

 Точка почти бесполезна

-->

 Поток запроса

- ->

 Поток ответа



Как локализовать потерю с помощью двух точек захвата

	Запрос виден на клиенте, но не виден на сервере	→	Потеря между клиентом и сервером	Проверяйте: сеть, firewall, маршрутизаторы, кабели, ACL, NAT
	Запрос пришел на сервер, но ответа нет	→	Проблема на сервере	Проверяйте: приложение, сервис, local firewall, лимиты, ресурсы
	Сервер отправил ответ, но клиент не получил	→	Проблема на обратном пути или у клиента	Проверяйте: обратный маршрут, асимметрия, потери, клиентский firewall
	Ответ виден на firewall, но не виден на клиенте	→	Проблема в клиентской сети	Проверяйте: маршрутизаторы провайдера, Wi-Fi, кабели, firewall клиента

Рекомендации по выбору точки захвата

- ✓ Начинайте с логического выбора по маршрутизации: `ip route get <dest>`
- ✓ Захватывайте на обеих сторонах проблемного участка
- ✓ Выбирайте интерфейс, через который реально идет трафик
- ✓ Учитывайте режим сети в VirtualBox и настройки Promiscuous Mode
- ✓ Сохраняйте PCAP с пояснениями: где, когда и почему сделан захват

Полезные команды

```
ip -br link
ip route get 10.0.0.10
sudo tcpdump -i enp0s3 -nn
sudo tcpdump -i enp0s3 -w capture.pcap
wireshark capture.pcap
```

💡 Правильная точка захвата = достоверные факты = быстрая диагностика

⚠ Неверная точка захвата может привести к ложным выводам!

2. Capture filter решает, какие пакеты попадут в файл

Capture Filter применяется во время захвата
tcpdump использует BPF (Berkeley Packet Filter) - язык
условий для отбора пакетов до записи

Фильтр уменьшает размер PCAP и снижает шум

Слишком узкий capture filter может безвозвратно
исключить важные пакеты

Если пакет не записан в PCAP, display filter потом его не
вернет

Для исследовательской диагностики безопаснее
начинать с умеренно широкого фильтра

Display filter скрывает лишнее, но не изменяет PCAP

Display Filter применяется после захвата в Wireshark
Он не удаляет пакеты из файла, а только управляет отображением

Пример display filter: `dns`

Пример для HTTP-порта: `tcp.port == 80`

Display filter безопасен для анализа уже записанного PCAP

Главное различие: `capture filter` ограничивает запись,
`display filter` ограничивает просмотр

Capture и display filters имеют разный синтаксис

Capture filter для порта: `tcp port 80`

Display filter для порта: `tcp.port == 80`

Capture filter для адреса: `host 192.168.10.20`

Display filter для адреса: `ip.addr == 192.168.10.20`

Capture filter работает до записи пакета, display filter работает после записи

Синтаксис BPF и синтаксис Wireshark нельзя смешивать

Базовые VRF-фильтры читаются как условия

host 192.168.30.10 - пакеты к узлу или от узла

net 192.168.30.0/24 - пакеты выбранной сети

port 53 - пакеты с портом источника или назначения 53

src host 192.168.10.5 - пакеты от конкретного
источника

dst host 192.168.30.10 - пакеты к конкретному
получателю

icmp, arp, udp, tcp и ip6 выбирают протокол или
семейство трафика

port, src port и dst port отвечают на разные вопросы

port 53 выбирает пакеты, где Source Port или Destination Port равен 53

src port 53 выбирает ответы, отправленные DNS-сервером с Source Port 53

dst port 53 выбирает запросы к DNS-серверу на Destination Port 53

tcp dst port 443 выбирает TCP-пакеты к HTTPS-порту назначения

src и dst помогают отличить запрос от ответа

Это особенно важно при анализе DNS, HTTP и клиентских ephemeral ports

Сложные фильтры нужно заключать в кавычки

Shell может интерпретировать скобки и специальные символы до запуска tcpdump

Безопасный пример: `sudo tcpdump -i enp0s3 'host 192.168.30.10 and tcp port 80'`

Пример со скобками: `sudo tcpdump -i enp0s3 'host 192.168.30.10 and (tcp port 80 or tcp port 443)'`

Кавычки делают фильтр одним аргументом команды tcpdump

В отчете нужно фиксировать фактически использованный capture filter

3. PCAP сохраняет пакеты для повторного анализа

PCAP - формат файла с сохраненным сетевым трафиком

Команда `sudo tcpdump -i enp0s3 -w network-test.pcap` записывает трафик в файл

При записи в файл `tcpdump` обычно не показывает подробности пакетов на экран

Команда `tcpdump -r network-test.pcap` читает ранее записанный файл

Wireshark открывает PCAP и показывает протоколы, поля, флаги и экспертные подсказки

PCAP позволяет вернуться к доказательствам после

Захват ограничивают по числу пакетов, времени или размеру

По числу пакетов: `sudo tcpdump -i enp0s3 -c 1000 -w test.pcap`

По времени: `sudo timeout 60 tcpdump -i enp0s3 -nn -w test.pcap`

По размеру и числу файлов: `sudo tcpdump -i enp0s3 -nn -C 100 -W 5 -w capture.pcap`

-C переключает файл при достижении заданного размера, -W ограничивает количество файлов

Точная трактовка размера и именования файлов зависит от версии tcpdump

Длинный захват без ограничения может заполнить

Snaplen определяет, сколько байт пакета сохраняется

Ключ -s задает размер сохраняемой части каждого пакета

Слишком маленький snaplen обрежет payload

Заголовки TCP могут быть видны, а HTTP-содержимое уже нет

Для полного пакета обычно используют -s 0 или достаточно большое значение

Пример: `sudo tcpdump -i enp0s3 -nn -s 0 -w full-capture.pcap`

Полный захват увеличивает размер файла и риск сохранения чувствительных данных

Полезные ключи `tcpdump` помогают читать пакет на месте

- n не преобразует IP-адреса в имена
- nn не преобразует IP-адреса и номера портов в имена
- v, -vv и -vvv показывают больше подробностей
- A выводит payload как ASCII, -X показывает Hex и ASCII
- e показывает L2-заголовок, включая MAC-адреса
- tttt выводит полные временные метки для сопоставления с журналами

ASCII и Hex-вывод могут раскрыть секреты прямо в терминале

Пример учебного HTTP: `sudo tcpdump -nn -A -i enp0s3 'tcp port 80'`

-A может показать cookie, Authorization header и параметры формы

-X показывает данные в Hex и ASCII, что полезно для низкоуровневого анализа

На производственном сервере такие ключи используют только при необходимости и с разрешением

Содержимое терминала тоже может попасть в скриншот, лог или отчет

PCAP нужно хранить как чувствительные диагностические данные

В PCAP могут быть IP-адреса, доменные имена, URL, cookies и учетные данные открытых протоколов

Файл нельзя оставлять в общедоступном /tmp или отправлять в общий чат без проверки

Пример каталога: `sudo install -d -m 700 /root/pcap`

Пример захвата: `sudo tcpdump -i enp0s3 -w /root/pcap/test.pcap`

Пример прав: `sudo chmod 600 /root/pcap/test.pcap`

Передача должна идти через защищенные способы, например SCP или SFTP

4. ARP показывает локальное разрешение IPv4 в MAC-адрес

ARP (Address Resolution Protocol) - протокол сопоставления IPv4-адреса и MAC-адреса в локальном сегменте

ARP-запрос спрашивает, кто имеет нужный IPv4-адрес

ARP-ответ сообщает MAC-адрес владельца IP

Команда `sudo tcpdump -i enp0s3 -e arp` показывает ARP-трафик и L2-заголовки

Команда `ip neigh show` показывает neighbor-таблицу Linux

ARP нужен до отправки IPv4-пакета соседу в той же L2-сети или шлюзу

Отсутствие ARP Reply обычно указывает на L2 или адресную проблему

Обычный IP firewall на iptables или nftables обычно не фильтрует ARP, потому что ARP не является IPv4-пакетом

Типичные причины: целевой узел выключен, интерфейс down, неверная VLAN или L2-разрыв

Также возможны неверная маска, отсутствие адреса на соседе или проблема виртуального коммутатора Port Security, ebtables, bridge-фильтрация или ARP-защита могут влиять на ARP

Если ARP-запрос уходит без ответа, первой проверяется L2-связность. VLAN и наличие адреса у

ARP Request и Reply нужно уметь читать в выводе

Пример request: ARP, Request who-has 192.168.10.1 tell 192.168.10.20

Это означает, что 192.168.10.20 ищет MAC-адрес шлюза 192.168.10.1

Пример reply: ARP, Reply 192.168.10.1 is-at 00:11:22:33:44:55

Это означает, что шлюз сообщил свой MAC-адрес
Состояния ip neigh REACHABLE и STALE обычно не означают аварию

Состояния INCOMPLETE или FAILED помогают подтвердить проблему разрешения соседнего адреса

DNS по умолчанию виден на UDP 53, но TCP 53 тоже важен

DNS (Domain Name System) - система преобразования доменных имен в IP-адреса

Обычные клиентские запросы чаще идут по UDP 53

TCP 53 применяется для передачи зон и некоторых крупных или повторных запросов

Фильтр `sudo tcpdump -i enp0s3 'port 53'` захватывает и UDP, и TCP 53

Ответ NOERROR означает успешное разрешение,

NXDOMAIN - имени не существует

SERVFAIL и REFUSED показывают ошибки или отказ на стороне DNS-сервера

Wireshark помогает разделить DNS-запросы от ответов

Display filter dns показывает весь DNS-трафик в PCAP

dns.flags.response == 0 показывает только запросы

dns.flags.response == 1 показывает только ответы

dns.flags.rcode != 0 показывает DNS-ответы с ошибками

Отсутствие ответа и отрицательный DNS-ответ
являются разными проблемами

Если запрос вообще не уходит, проверяются

resolv.conf, systemd-resolved, NetworkManager или
firewall

Не весь современный DNS виден как port 53

DoT (DNS over TLS) обычно использует TCP 853

DoH (DNS over HTTPS) использует HTTPS и обычно идет через TCP 443

При DoH DNS-запрос похож на обычный HTTPS-трафик к известному endpoint

Локальный клиент может обращаться к systemd-resolved через loopback, а внешний запрос отправит другой процесс

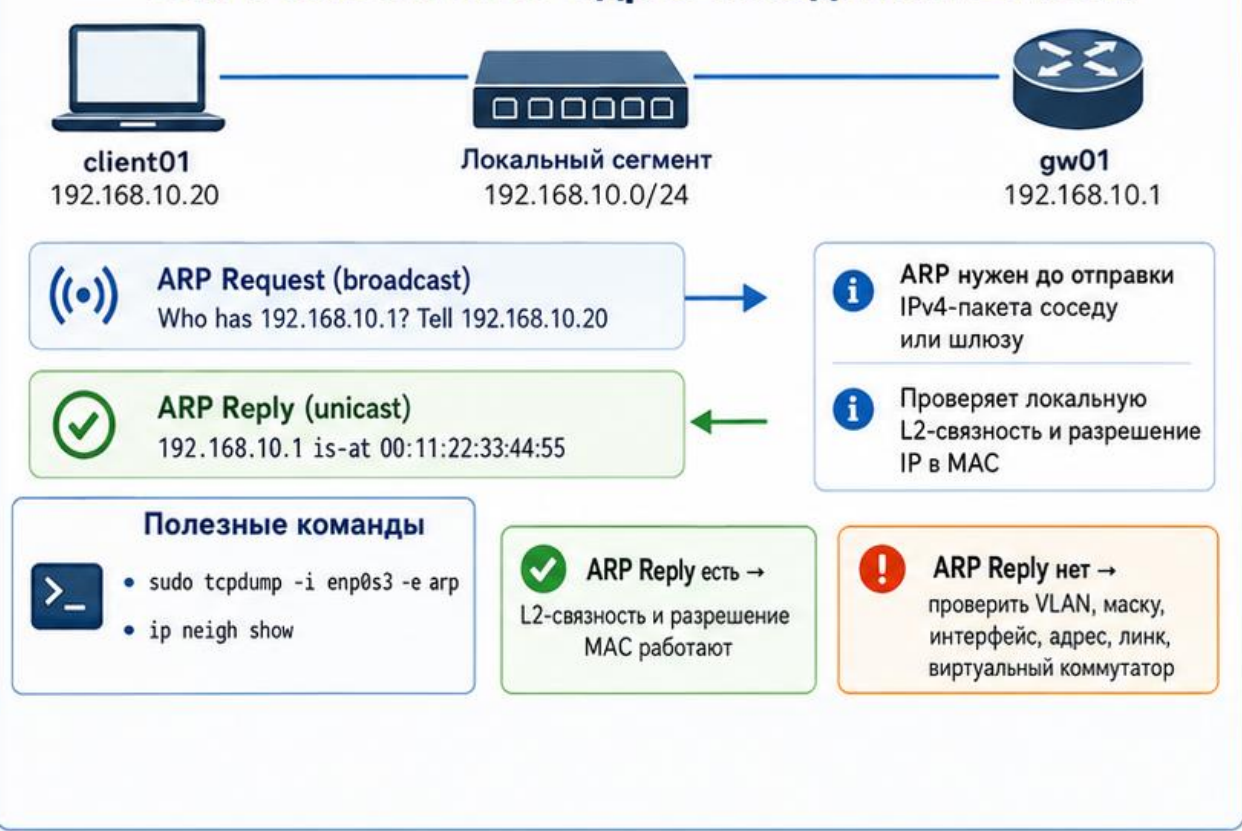
Отсутствие пакетов на port 53 не всегда доказывает отсутствие DNS-запросов

Для базовой лаборатории обычный DNS по UDP/TCP

Схема: ARP и DNS в диагностике подключения

Сначала проверяем IP→MAC в локальном сегменте, затем имя→IP

ARP: есть ли MAC-адрес соседа или шлюза?



DNS: разрешается ли имя в IP?



Как читать симптомы

?	По IP доступ есть, по имени нет	→	Вероятна DNS-проблема
!	Нет ARP Reply к шлюзу	→	L2/VLAN/адресная проблема
!	DNS Query уходит, но Response нет	→	DNS-сервер недоступен или трафик блокируется
✗	DNS отвечает NXDOMAIN	→	Неверное имя или отсутствует DNS-запись

5. TCP handshake показывает установку соединения

TCP (Transmission Control Protocol) - транспортный протокол с установлением соединения и контролем доставки

Three-way handshake состоит из SYN, SYN/ACK и ACK
SYN отправляет клиент, чтобы начать соединение
SYN/ACK отвечает сервер, если порт доступен и соединение принимается

ACK клиента завершает установку TCP-соединения

Handshake доказывает TCP-доступность порта, но не доказывает корректный HTTP-ответ

SYN-сценарии помогают отличить открытый порт, reject и drop

Открытый порт: SYN, затем SYN/ACK, затем ACK

Закрытый порт или активный reject: SYN, затем RST/ACK или ICMP-сообщение

Молчаливый drop: SYN, затем повторные SYN без ответа

RST не всегда означает, что именно приложение отвергло соединение

Для начального SYN наиболее вероятны закрытый порт или активный reject

Один PCAP нужно сопоставлять с firewall, ss, журналами и точкой захвата

Штатное завершение TCP идет через FIN и ACK

FIN (Finish) - TCP-флаг штатного завершения передачи в одном направлении

Упрощенная последовательность: Client -> Server
FIN,ACK

Затем Server -> Client ACK

После этого Server -> Client FIN,ACK

Завершает обмен Client -> Server ACK

Так как каждое направление закрывается отдельно, часто говорят о четырехэтапном завершении

RST и timeout означают разные диагностические ситуации

RST (Reset) - реально полученный TCP-сегмент резкого сброса соединения

RST означает активный ответ: узел или устройство явно отвергло соединение

Timeout не является TCP-пакетом или флагом

Timeout - результат ожидания приложения или диагностического инструмента, когда ответ не пришел вовремя

Timeout может возникнуть из-за firewall drop, потери маршрута, недоступного узла или обратного маршрута

Одного timeout недостаточно для точного

Retransmission и Duplicate ACK являются выводом аналитики Wireshark

Retransmission - повторная отправка TCP-сегмента

Duplicate ACK - повторное подтверждение одного и того же места в TCP-потоке

В Wireshark это TCP analysis flags, а не отдельные TCP-флаги в заголовке

Wireshark делает вывод по Sequence Number, Acknowledgment Number и времени

Массовые повторы могут указывать на потери, задержки, MTU, перегрузку сервера или качество канала

Единичное событие нужно подтверждать контекстом.

Wireshark может ошибиться, если захват неполный

Захват мог начаться в середине TCP-соединения

Часть пакетов могла не попасть в capture из-за фильтра или перегрузки

Пакеты могли быть собраны в нескольких точках и объединены неидеально

Packet offloading может изменить вид локального захвата

Подтверждать retransmission нужно через sequence numbers, ACK numbers, интервалы времени и второй захват

Пакетный анализ дает наблюдение, но первопричину

Offloading может искажать локальный захват на Linux

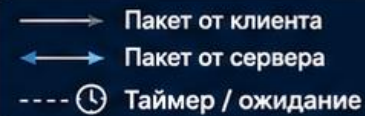
TCP Segmentation Offload, Generic Segmentation Offload и Generic Receive Offload меняют момент сегментации пакетов

Checksum Offload может привести к сообщению Wireshark о неправильной checksum в локальном PCAP
На физическом проводе пакет при этом может быть корректным

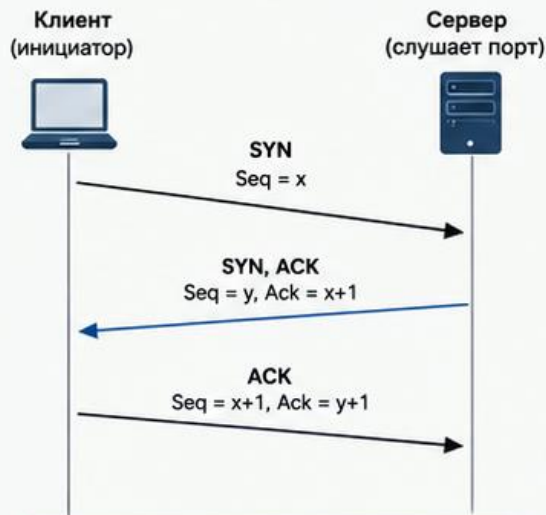
Команда `sudo ethtool -k enp0s3` показывает включенные offloading-функции интерфейса
Отключать offloading на рабочем сервере без необходимости не следует

Диаграмма: TCP SYN, FIN, RST и timeout

Флаги TCP помогают понять состояние соединения и причину проблемы



1. Установка соединения (TCP 3-way handshake)



✓ Соединение установлено (ESTABLISHED)
Данные могут передаваться в обе стороны

Как диагностировать

- SYN ушёл, SYN/ACK вернулся → путь к серверу открыт
- SYN ушёл, ответа нет → проблема на пути к серверу
- SYN/ACK пришёл, ACK не ушёл → проблема на клиенте
- Используйте: tcpdump, ss -nt, traceroute

2. Корректное завершение (FIN/ACK)



✓ Соединение корректно закрыто (CLOSED)

Как диагностировать

- FIN от одной стороны → она завершает передачу
- ACK подтверждает FIN
- FIN от другой стороны → её очередь пуста
- Последний ACK → соединение закрыто

3. Принудительное закрытие (RST)



! Соединение разорвано немедленно
Ошибки: Connection refused, Connection reset by peer

Как диагностировать

- RST приходит сразу → порт закрыт или служба не запущена
- RST во время соединения → соединение сброшено приложением, firewall или системой
- Ищите источник RST с помощью tcpdump

4. Timeout (таймаут)



Как диагностировать

- Нет ответа на SYN → фильтрация, потеря пакетов или маршрут отсутствует
- Проверьте: firewall, маршруты, ACL, проблему сети
- Используйте: traceroute, ping, mtr, tcpdump

Ключевые TCP флаги

- SYN** – синхронизация номеров последовательности, запрос на соединение
- ACK** – подтверждение получения (Ack = номер следующего ожидаемого байта)
- FIN** – завершение передачи данных (больше нечего отправлять)
- RST** – сброс соединения (ошибка, недопустимое состояние или политика)
- PSH/URG** – служебные флаги (данные немедленно / срочные данные)

Полезные команды

```
# Захват трафика
sudo tcpdump -ni any tcp port 80
# Просмотр соединений
ss -nt state all
# Проверка маршрута
ip route get <IP-адрес>
```

Что смотреть в Wireshark

- Флаги в колонке Info: [S],[S.],[.],[F.],[R],[R.]
- Номера последовательности (Seq) и подтверждения (Ack)
- Window size, retransmission, duplicate ACK
- RTT (Round Trip Time) и время ожидания

Подсказка

Пакетный анализ показывает фактическое поведение сети, а не то, что «должно быть». Два захвата (клиент + сервер) дают полную картину!

6. HTTP можно читать как прикладный протокол поверх TCP

HTTP (HyperText Transfer Protocol) - прикладной протокол передачи web-запросов и ответов

В открытом HTTP видны метод, путь, Host header и код ответа

Пример запроса: GET /index.html HTTP/1.1

Пример заголовка: Host: web01.company.test

Пример ответа: HTTP/1.1 200 OK

Wireshark display filter http помогает быстро выделить HTTP-события

HTTP-коды помогают разделить сеть от приложения

TCP handshake есть, но HTTP Request отсутствует:

проблема клиента или приложения

HTTP Request есть, но ответа нет: проверяется сервер или обратный путь

HTTP 200 означает успешный ответ приложения

HTTP 404 означает, что сервер доступен, но ресурс не найден

HTTP 500 означает, что сервер ответил, но приложение завершилось ошибкой

HTTP 301 или 302 означает перенаправление и не всегда является ошибкой

HTTPS скрывает содержимое, но часть метаданных остается видимой

HTTPS - HTTP поверх TLS-шифрования

Обычно остаются видны IP-адреса, TCP-порты, время, размеры пакетов и TCP handshake

В обычных TLS-сценариях часто видны параметры handshake и иногда SNI

Точный набор открытых полей зависит от версии TLS и используемых расширений

В TLS 1.3 часть handshake-сообщений защищена сильнее, чем в старых версиях

Не следует обещать, что имя сайта или сертификат всегда будут видны пассивному наблюдателю

Расшифровка HTTPS требует контролируемой и разрешенной среды

Приватный ключ сервера не является универсальным способом расшифровки современного TLS

Perfect Forward Secrecy через ephemeral Diffie-Hellman не позволяет расшифровать прошлый сеанс только приватным ключом

В учебной среде возможен SSLKEYLOGFILE на контролируемом клиенте

Также используют отладочный reverse proxy, TLS-терминацию на тестовом прокси или серверные журналы

Расшифровка допустима только в лаборатории или

Break/Fix-сценарии учат читать отрицательные признаки

Неверная VLAN или выключенный сосед: ARP Request без Reply

Неверная DNS-запись: DNS Response NXDOMAIN

DNS-сервер недоступен: DNS Query без Response

Закрытый TCP-port: SYN -> RST/ACK

Firewall Drop: повторные SYN без ответа

Потеря пакетов: retransmission и duplicate ACK в контексте последовательностей

Успешный и аварийный web-обмен выглядят по-разному

Успешный web: TCP handshake, HTTP GET и HTTP 200

Несуществующая страница: TCP есть, HTTP 404

Ошибка приложения: TCP есть, HTTP 500

Штатное закрытие: FIN/ACK-обмен после передачи данных

Резкий сброс: RST вместо нормального завершения

Диагностический вывод должен ссылаться на конкретный пакет или группу пакетов

Отчет по PCAP должен фиксировать условия захвата

В отчете указывается симптом и проверяемая гипотеза

Обязательно фиксируются capture point, интерфейс и capture filter

Нужно указать время или интервал захвата

Для доказательства указывают номер пакета или диапазон пакетов в Wireshark

Наблюдение формулируется технически: SYN без ответа, NXDOMAIN, RST/ACK или HTTP 500

Пакетный захват подтверждает сетевое поведение, но не всегда сам доказывает первопричину

Пример строки отчета по захвату

Симптом: client01 не открывает

http://web01.company.test

Capture Point: client01, интерфейс enp0s3, фильтр 'host 192.168.10.20 and tcp port 80'

Наблюдение: пакеты 15, 18 и 21 являются повторными SYN без SYN/ACK

Вывод: клиент отправляет запрос, но ответа от сервера не получает

Дополнительная проверка: второй захват на web01 и проверка firewall

После исправления: handshake завершен. HTTP GET

Итоги лекции

Пакетный анализ показывает фактическое сетевое поведение между узлами

Точка захвата и интерфейс определяют, какие пакеты будут видны

Capture filter ограничивает запись, display filter ограничивает просмотр

PCAP нужно ограничивать по объему и хранить как чувствительные данные

ARP, DNS, TCP и HTTP дают разные диагностические признаки

RST, timeout, retransmission и HTTPS требуют

Контрольные вопросы

Почему один захват на клиенте может быть недостаточен для вывода о причине проблемы?
Что показывает `ip route get` и чего эта команда не доказывает?

Чем `capture filter` отличается от `display filter`?

Почему сложные BPF-фильтры лучше заключать в кавычки?

Почему обычный IP firewall обычно не является первой причиной отсутствия ARP Reply?

Чем RST отличается от timeout?

Почему `retransmission` в Wireshark является